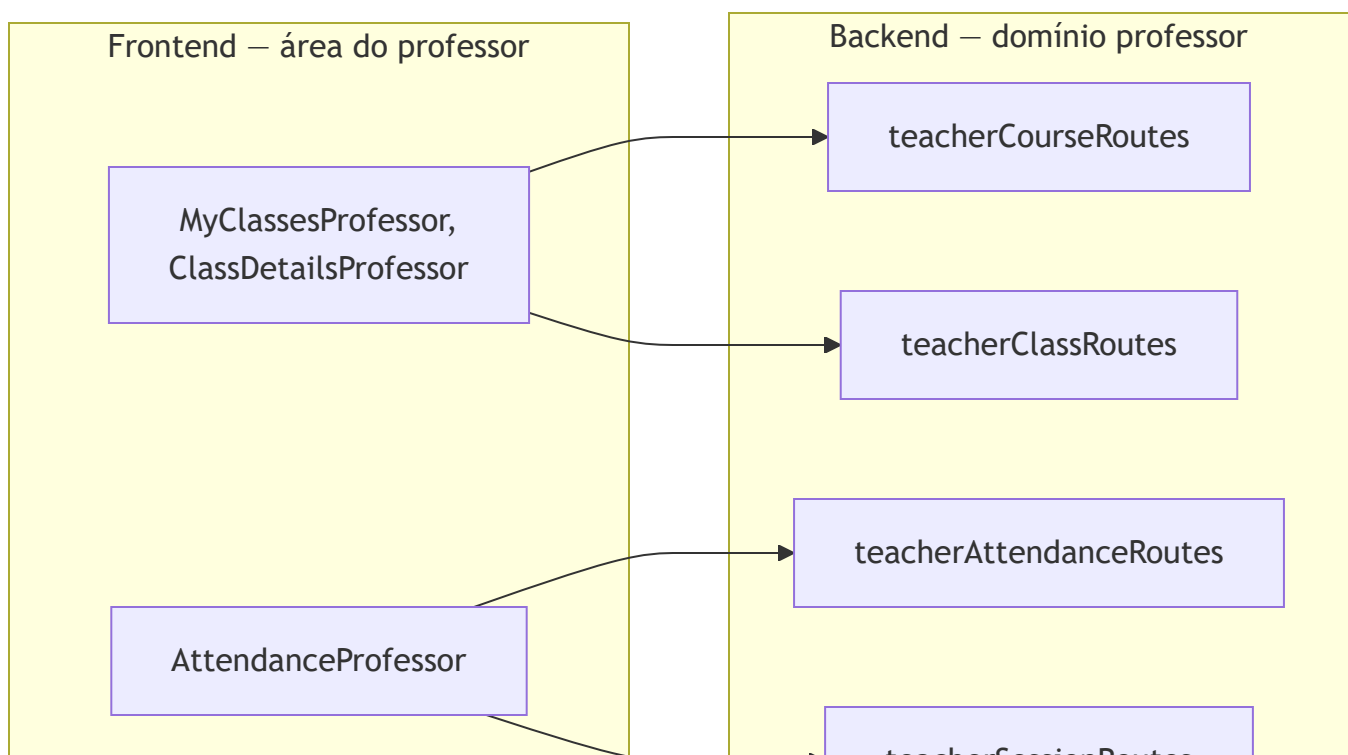
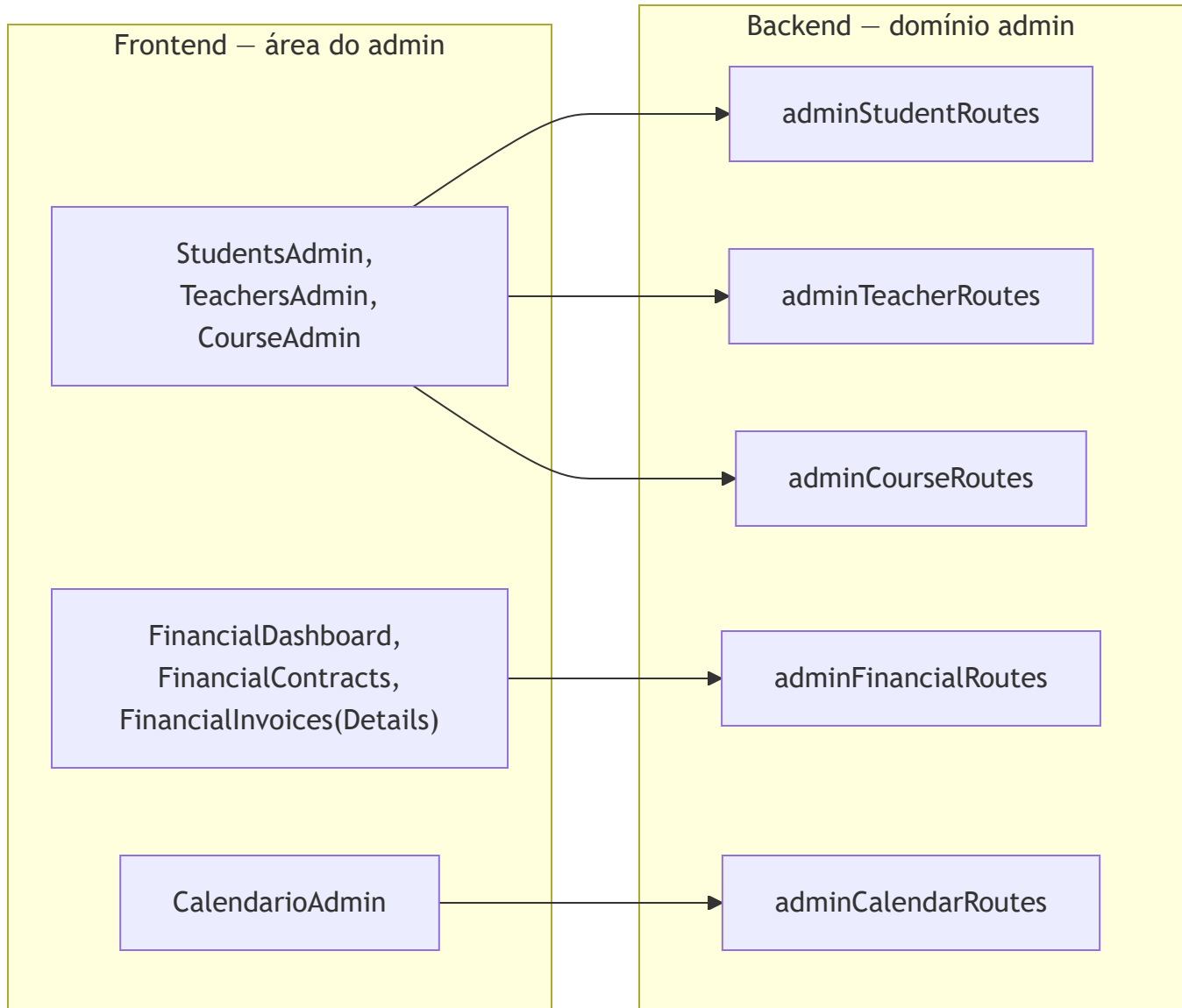
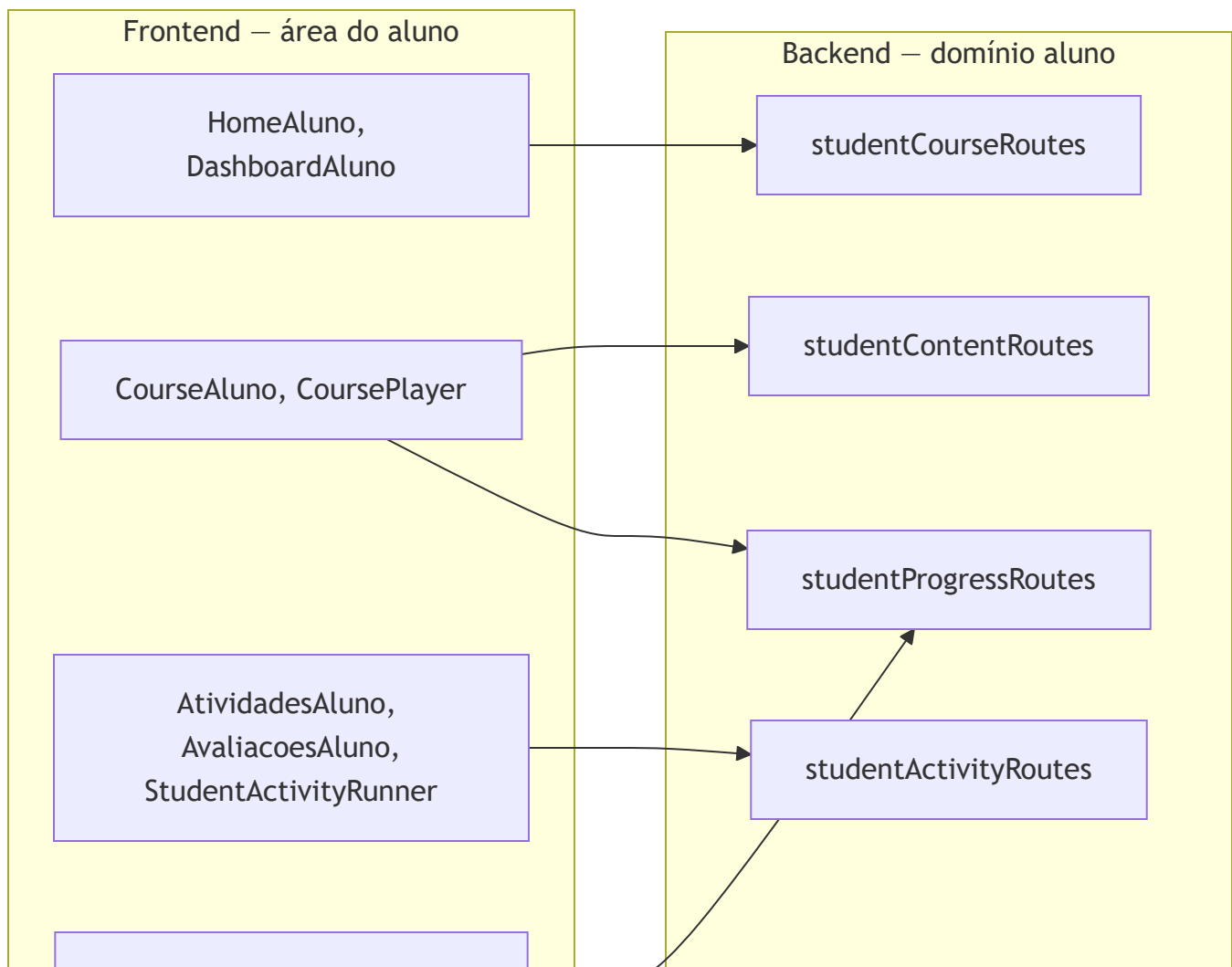
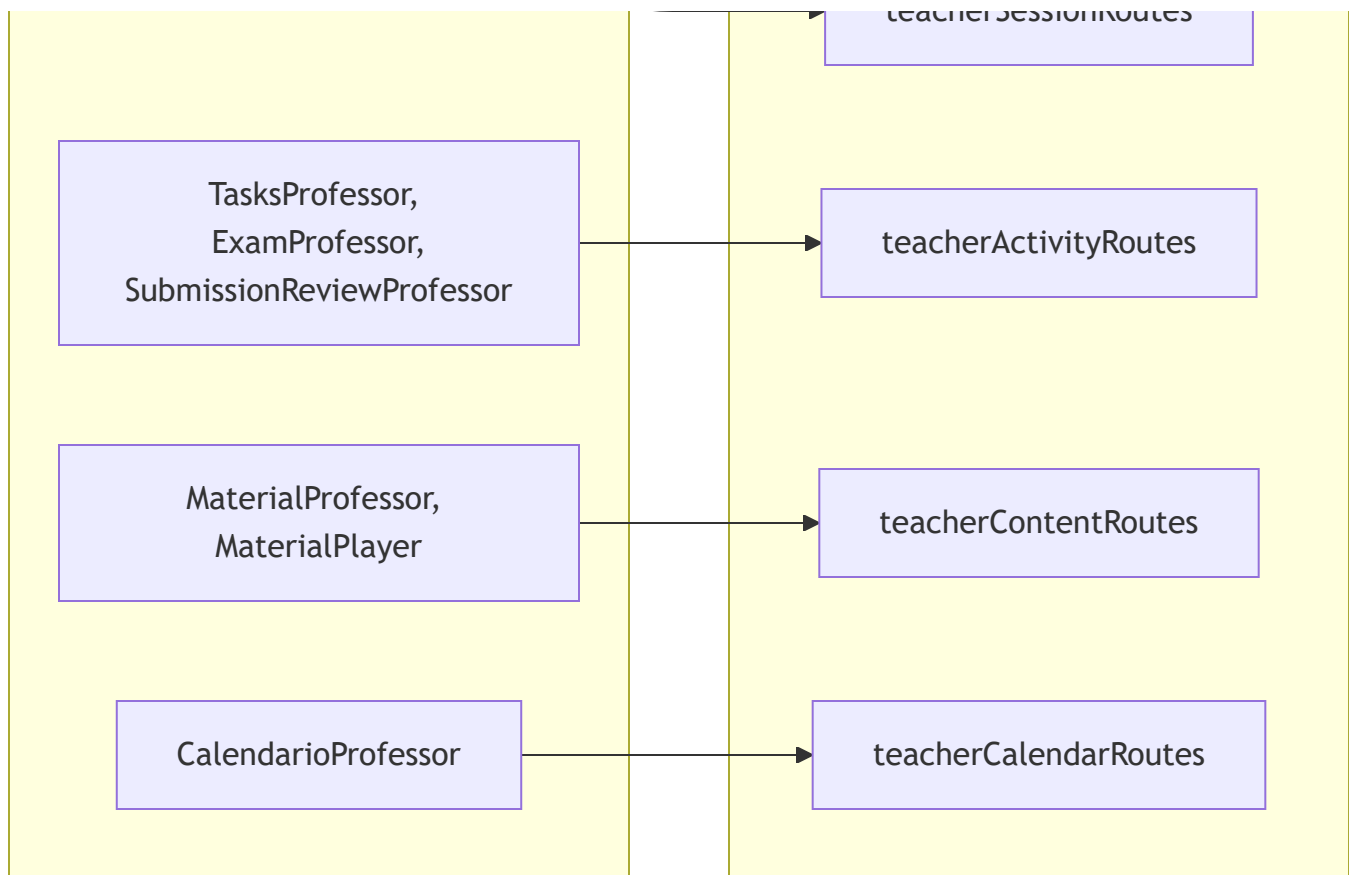
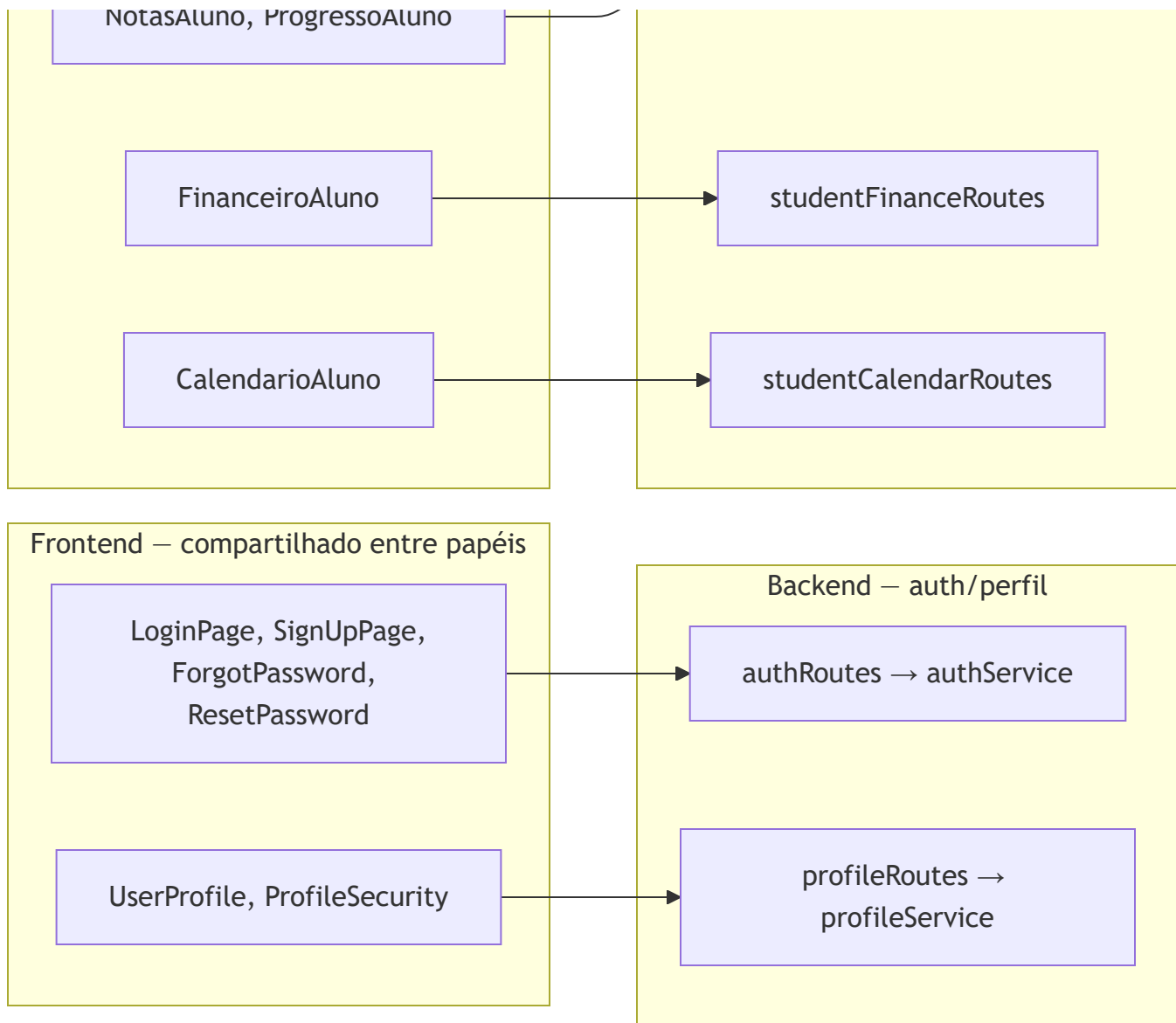


Diagrama — Dependências entre Módulos (Frontend ↔ Backend)

Diferente de `backend-architecture.md` (camadas dentro do backend), este diagrama mostra o acoplamento **entre** as áreas do frontend e os domínios de backend que elas realmente consomem — útil para responder "se eu mudar este service do backend, quais telas eu preciso testar?".







Que problema este diagrama esclarece

Em um projeto onde frontend e backend evoluíram juntos por um bom tempo, é fácil uma rota de backend virar código morto sem ninguém perceber, ou uma tela do frontend depender de um endpoint que já foi renomeado. Este diagrama é o mapa de "quem chama quem" entre as duas metades do sistema — a referência para avaliar o impacto real de alterar qualquer rota.

Como interpretar os elementos

- Cada seta representa uma dependência de chamada HTTP real, confirmada lendo o componente do frontend (`apiFetch("/api/...")`) e conferindo contra as rotas montadas em `server.js` .
- Os agrupamentos (`FE_ALUNO` , `BE_ALUNO` , etc.) espelham a mesma organização por papel usada em todo o resto do sistema — deliberadamente consistente com `backend-architecture.md` e `Router.jsx` .

- Este diagrama **não** mostra os módulos administrativos de conteúdo/atividade/calendário do lado admin porque o admin não tem telas próprias de conteúdo/atividade — só gerencia calendário institucional e financeiro diretamente; conteúdo e atividades são geridos pelo professor.
- Componentes compartilhados entre papéis (`UserProfile` , `ProfileSecurity` , as páginas de auth) aparecem em um subgrafo à parte porque são literalmente os mesmos arquivos React usados nas três áreas, não uma cópia por papel.

Que decisões arquiteturais este diagrama evidencia

- **Simetria estrutural entre aluno e professor:** os dois lados têm o mesmo número de domínios de rota (cursos, conteúdo, atividades, calendário), refletindo que o produto foi desenhado com os dois papéis em mente desde o início do módulo de atividades/conteúdo, não um estendido a partir do outro.
- **Admin não duplica rotas de aluno/professor:** quando o admin precisa ver dados de curso/turma, o frontend consulta os mesmos endpoints administrativos (`adminCourseRoutes` , etc.), não uma cópia dos endpoints de professor com checagem de papel diferente — evita uma segunda implementação da mesma consulta.
- **Todo domínio de frontend aponta para exatamente um domínio de backend equivalente:** não há uma tela do frontend que precise orquestrar 3+ domínios de backend não relacionados para renderizar — cada área de tela tem uma "casa" clara no backend, evidência de que o design de API acompanhou o design de tela, não o contrário.

Trade-offs que este mapeamento representa

- **Um domínio de backend por conceito de tela** (em vez de endpoints mais genéricos e reutilizáveis) significa mais arquivos de rota (21 no total), mas cada um pequeno e de responsabilidade única — mais fácil de auditar individualmente, mais arquivos para navegar como conjunto.
- **Acoplamento direto tela→rota:** uma mudança de contrato em `studentProgressRoutes` (por exemplo, remover um campo da resposta) tem exatamente dois consumidores conhecidos (`NotasAluno` , `ProgressoAluno`) — bom para avaliar impacto, mas significa que não há uma camada de abstração no frontend que isolaria as telas de mudanças de contrato do backend (o `services/` do frontend hoje é fino, repassa a resposta quase crua).

Como este diagrama deve evoluir

Ao criar uma nova tela, identifique primeiro se ela se encaixa em um domínio de backend existente (adicionar ao router correspondente) antes de criar um novo — este diagrama deve continuar mostrando um mapeamento limpo de "uma área de tela, um domínio de backend", não many-to-many. Se uma tela realmente precisar combinar dados de múltiplos domínios que hoje não se falam (como o calendário já faz deliberadamente), documente essa exceção explicitamente, seguindo o padrão de agregador já estabelecido em `calendar-flow.md`, em vez de fazer o frontend orquestrar múltiplas chamadas ad-hoc.